Figure 26.11 The Seq UI. Logs are presented as a list. You can view the structured logging details of individual logs, view analytics for logs in aggregate, and search by log properties.

ASP.NET Core supports structured logging by treating each captured parameter from your message format string as a key-value pair. If you create a log message using the following format string,

```
_log.LogInformation("Loaded {RecipeCount} recipes", Recipes.Count);
```

the Seq logging provider creates a `RecipeCount` parameter with a value of `Recipes .Count`. These parameters are added as properties to each structured log, as you can see in figure 26.11.

Structured logs are generally easier to read than your standard-issue console output, but their real power comes when you need to answer a specific question. Consider the problem from before, where you see this error:

```
Could not find recipe with id 3245
```

You want to get a feel for how widespread the problem is. The first step would be to identify how many times this error has occurred and to see whether it’s happened to any other recipes. Seq lets you filter your logs, but it also lets you craft SQL queries to analyze your data, so finding the answer to the question takes a matter of seconds, as shown in figure 26.12.